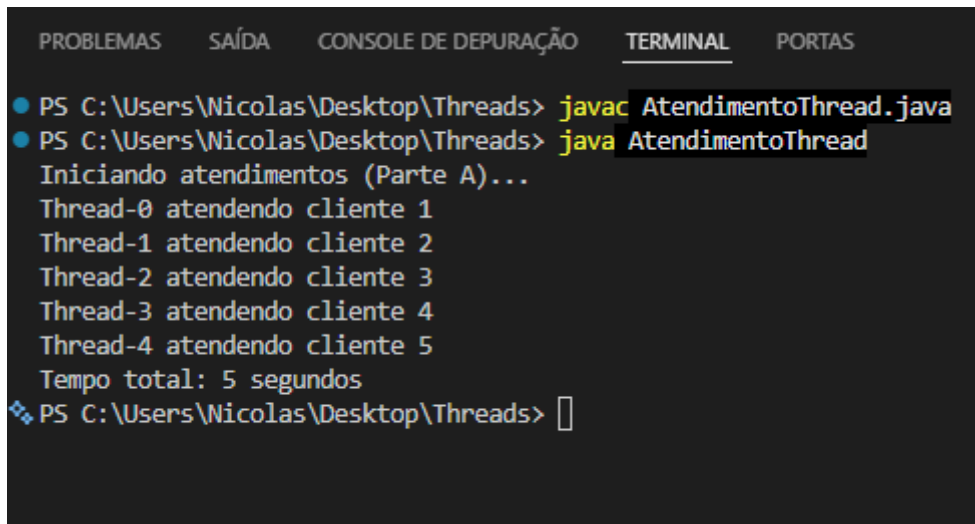


Roteiro 1 – Threads

Aluno: Nicolas Almeida Prado da Silva

Parte A - Forma Clássica #1 (extends Thread)

- **Pergunta:** O tempo total ficou perto de 1s ou de 5s com 5 atendimentos? Por quê?
- **Resposta:** Conforme a execução no terminal, o tempo total ficou em 5 segundos. Isso ocorreu porque o método `join()` foi chamado dentro do laço de repetição logo após o `start()`. O `join()` bloqueia a thread principal até que a thread iniciada termine sua execução. Como cada thread tem um atraso de 1 segundo (`Thread.sleep(1000)`), as 5 threads acabaram sendo executadas de forma sequencial, somando 5 segundos totais, em vez de executarem em paralelo.



```
PROBLEMAS  SAÍDA  CONSOLE DE DEPURAÇÃO  TERMINAL  PORTAS
PS C:\Users\Nicolas\Desktop\Threads> javac AtendimentoThread.java
PS C:\Users\Nicolas\Desktop\Threads> java AtendimentoThread
Iniciando atendimentos (Parte A)...
Thread-0 atendendo cliente 1
Thread-1 atendendo cliente 2
Thread-2 atendendo cliente 3
Thread-3 atendendo cliente 4
Thread-4 atendendo cliente 5
Tempo total: 5 segundos
PS C:\Users\Nicolas\Desktop\Threads> 
```

Parte B - Forma Clássica #2 (implements Runnable)

- **Pergunta:** Qual das duas classes (Parte A ou B) você poderia fazer herdar de outra classe hoje?
- **Resposta:** A classe da Parte B (`AtendimentoRunnable`). Como a linguagem Java não permite herança múltipla, a classe da Parte A já está "presa" à classe `Thread`. A classe da Parte B, por utilizar a interface `Runnable`, fica totalmente livre para herdar de qualquer outra classe, permitindo uma arquitetura mais flexível.

```

PS C:\Users\Nicolas\Desktop\Threads> java AtendimentoRunnable
Iniciando atendimentos (Parte B)...
atendendo cliente 1
PS C:\Users\Nicolas\Desktop\Threads>

```

Parte C - O Problema de Escala

- **Pergunta 1:** Por que criar uma thread de SO é mais caro do que criar um objeto comum em Java?
- **Resposta 1:** Porque cada thread nativa reserva tipicamente entre 512KB e 1MB de memória de pilha, além de depender da criação de estruturas de controle e gerenciamento por parte do próprio kernel do Sistema Operacional.
- **Pergunta 2:** O que esse limite sugere sobre usar 1 thread por requisição em um servidor web?
- **Resposta 2:** Sugere que a arquitetura de alocar uma thread inteira do Sistema Operacional para cada requisição não é sustentável nem escalável em cenários de alta concorrência. Se um servidor web receber milhares de conexões simultâneas, ele esgotará rapidamente a memória do processo, resultando na falha da aplicação devido à incapacidade de criar novas threads nativas.

```

PS C:\Users\Nicolas\Desktop\Threads> javac ProblemaEscala.java
PS C:\Users\Nicolas\Desktop\Threads> java ProblemaEscala
Iniciando teste de limite do Sistema Operacional...
Finalizou as 10.000 threads!

```

Parte D - ExecutorService

- **Pergunta:** Com 4 threads atendendo 10 clientes, o tempo total ficou perto de 1s, 2s ou 3s?
- **Resposta:** Ficou perto de 3 segundos, exatamente como comprovado na execução. Como o pool possui um limite fixo de 4 threads reais do SO, e cada atendimento tem a duração de 1 segundo, o fluxo ocorreu em lotes: 4 clientes foram atendidos no primeiro segundo, mais 4 clientes no segundo segundo, e os 2 últimos clientes no terceiro segundo (totalizando 3 segundos). Também é visível no log que os nomes das threads (ex: pool-1-thread-1) se repetiram, comprovando que o pool as reaproveitou.

```

PS C:\Users\Nicolas\Desktop\Threads> javac AtendimentoPool.java
PS C:\Users\Nicolas\Desktop\Threads> java AtendimentoPool
Iniciando pool de threads (Parte D)...
pool-1-thread-4 atendendo cliente 4
pool-1-thread-1 atendendo cliente 1
pool-1-thread-2 atendendo cliente 2
pool-1-thread-3 atendendo cliente 3
pool-1-thread-1 atendendo cliente 5
pool-1-thread-3 atendendo cliente 6
pool-1-thread-4 atendendo cliente 7
pool-1-thread-2 atendendo cliente 8
pool-1-thread-4 atendendo cliente 9
pool-1-thread-3 atendendo cliente 10
Tempo total: 3 segundos
PS C:\Users\Nicolas\Desktop\Threads>

```

Parte E - Virtual Threads

- **Pergunta:** Uma Virtual Thread é uma thread de Sistema Operacional? Se não, o que ela é?
- **Resposta:** Não. As Virtual Threads são threads gerenciadas inteiramente pela própria JVM, não possuindo um mapeamento de 1 para 1 com as threads nativas do Sistema Operacional. Elas são extremamente leves e ocupam uma fração mínima de memória. Por debaixo dos panos, milhões de Virtual Threads podem rodar sendo alternadas rapidamente sobre um número pequeno de threads reais do SO.

```

PS C:\Users\Nicolas\Desktop\Threads> javac AtendimentoVirtual.java
PS C:\Users\Nicolas\Desktop\Threads> java AtendimentoVirtual
Iniciando Virtual Threads (Parte E)...
As 100.000 tarefas foram processadas sem estourar a memória (Sem OutOfMemoryError)!
PS C:\Users\Nicolas\Desktop\Threads>

```

Exercícios e Checklist de Entrega

- **Trocar o pool fixo por `newCachedThreadPool()` - o comportamento muda?** Sim, muda drasticamente. O `newCachedThreadPool()` cria novas threads nativas sob demanda e reaproveita as ociosas. Se submetermos 10 tarefas simultâneas, ele criará 10 threads na hora, executando tudo em paralelo e finalizando em aproximadamente 1 segundo, em vez dos 3 segundos vistos no pool fixo.
- **Imprimir `Thread.currentThread()` na Parte E - é uma Virtual Thread?** Sim. Se essa impressão for feita, a saída no console mostrará explicitamente o identificador indicando que se trata de uma thread virtual, diferentemente do padrão das threads clássicas.
- **Explicar processo x thread com a analogia do guichê de atendimento:** O processo pode ser comparado ao prédio da agência

bancária inteira (comporta o endereço, espaço físico e recursos básicos de infraestrutura). As threads são os guichês de atendimento individuais instalados dentro da agência. Vários guichês (threads) operam simultaneamente e de forma independente, mas todos compartilham o mesmo espaço e os recursos da agência (processo) para atender aos clientes.

- **Escolher, com justificativa, a abordagem ideal para um servidor com milhares de conexões:** A abordagem ideal é o uso de Virtual Threads (Norma atual do Java 21+). A justificativa é a leveza extrema. Em uma abordagem de "uma thread nativa por conexão", o servidor esgotaria sua memória rapidamente. Com as Virtual Threads, o servidor consegue sustentar milhares de conexões simultâneas de forma barata para a máquina física, não estourando os limites do Sistema Operacional.